

SVX

SystemVerilog Extension Library

Examples Catalog

Version 2.1.2 beta

October 2026

Mark Glasser

Apache License 2.0

Contents

1	Containers	2
1.1	Vector	2
1.2	Deque	2
1.3	Queue	2
1.4	Map	3
2	Algorithms	4
2.1	Algo	4
3	Types	5
3.1	Type Check	5
4	Lexer	6
4.1	RPN Calculator	6
5	Use Models	7
5.1	List	7
5.2	Map — Basic Use	7
5.3	Map — Polymorphic Container	8
5.4	Algo with Range	8
6	Applications	9
6.1	Sparse Memory	9
6.2	Device Memory Map	9
6.3	Priority Queue Job Scheduler	10

Chapter 1: Containers

The container examples demonstrate the four core generic containers provided by SVX: `vector`, `deque`, `queue`, and `map`. Each example is self-contained and focuses on the essential operations for that container.

1.1 Vector

Source: `examples/containers/vector`

Demonstrates the two distinct ways to grow a vector. In the first approach, elements are written past the current end of the vector using `write()`; the vector extends itself automatically to accommodate the new index. In the second approach, `appendc()` adds new elements one at a time to the back, also growing the vector. After population the example performs random-access reads and a targeted write, then traverses the vector by index to print its contents.

SVX features exhibited:

- `vector#(T,P)`
- `int32_traits`

1.2 Deque

Source: `examples/containers/deque`

Demonstrates double-ended queue operations. Elements are pushed onto both the back and the front of the deque, then drained from the front with `pop_front()`. A second pass re-populates the deque and shows that `read()`, inherited from the vector base class, can be used for indexed access even on a deque instance. The deque is then reversed in place and printed again.

SVX features exhibited:

- `deque#(T,P)`
- `int32_traits`

1.3 Queue

Source: `examples/containers/queue`

Demonstrates storing user-defined class objects in a queue. The example defines a packet class with a randomized destination byte and a 16-byte payload stored in a vector. A static vector of valid destinations is shared across all packet instances and populated once at setup time.

Ten packets are created via a static factory function, which selects a random destination from the shared table and fills the payload with random bytes. The packets are put into a queue and then retrieved in FIFO order. The example illustrates the use of `class_traits#()` as the policy parameter required when storing class handles in a container.

SVX features exhibited:

- `queue#(T,P)`
- `vector#(T,P)`
- `class_traits#(T)`
- `uint8_traits`

1.4 Map

Source: `examples/containers/map`

Demonstrates a string-keyed map whose values are polymorphic class handles. Three concrete classes (`str_obj`, `real_obj`, `int_obj`) each extend an abstract base class that declares a virtual `to_string()` method. One instance of each is created and inserted into the map under a descriptive key.

The example then performs direct key-based retrieval and calls `to_string()` on each result. A final traversal using the `first()/next()` idiom on the map itself prints all entries, illustrating map iteration without an explicit iterator object.

SVX features exhibited:

- `map#(KEY,T,P)`
- `class_traits#(T)`

Chapter 2: Algorithms

The algorithm examples demonstrate the `algo#()` class, which provides a set of higher-order, iterator-based algorithms. Each algorithm accepts an iterator and a predicate or functor, operating generically over any container that exposes the forward iterator interface.

2.1 Algo

Source: `examples/algo/algo`

Demonstrates the core algorithms in `algo#()` applied to a small vector of bytes. A custom predicate class (extending `predicate#()`) tests whether each element is less than 0x0f. A custom functor class (extending `fcn#()`) prints each element.

The example runs five algorithms against the vector: `all_of`, `none_of`, `any_of`, `count`, and `for_each`. The results of the predicate-based queries are printed as boolean strings, and `for_each` prints the full vector contents.

SVX features exhibited:

- `algo#(T,P)`
- `predicate#(T)`
- `fcn#(T)`
- `vector#(T,P)`
- `list_iterator#(T,P)`
- `uint8_traits`

Chapter 3: Types

The types examples demonstrate SVX's compile-time type classification and trait-checking facilities.

3.1 Type Check

Source: `examples/types/type_check`

Demonstrates compile-time type trait assertions via the `check_trait` macro. The example class is parameterized by a type `T` and its associated traits class `P`. Three `check_trait` invocations at class elaboration time assert that the type is integral, is two-state, and is not a string. If any assertion fails, elaboration terminates with a fatal message.

At runtime the `show()` method uses `typeid#(T)` to query and print the same three trait values, confirming what was already verified statically. The example is instantiated with `uint16_t` and `uint16_traits`.

SVX features exhibited:

- `typeid#(T)`
- `void_traits`
- ``check_trait` (macro)

Chapter 4: Lexer

The lexer example demonstrates `lexer_core` in a realistic application context: a reverse Polish notation (RPN) calculator.

4.1 RPN Calculator

Source: `examples/lexer`

Implements a reverse Polish notation (RPN) calculator using the `lexer_core` tokenizer and a stack. The lexer tokenizes an input string into integer operands, floating-point operands, and arithmetic operators (+, -, *, /). The calculator maintains a stack of `triple#()` items, where each item carries the token type, an integer value, and a real value. For each token the calculator either pushes an operand onto the stack or pops two operands, performs the operation, and pushes the result.

The example evaluates six expressions covering integer arithmetic, floating-point arithmetic, mixed-type coercion, and integer and floating-point division by zero. The `triple#()` class is used as a discriminated union, with the first field carrying the token type and the second and third fields carrying integer and real values respectively.

SVX features exhibited:

- `lexer_core`
- `stack#(T,P)`
- `triple#(A,B,C)`
- `class_traits#(T)`

Chapter 5: Use Models

The use model examples are longer, more detailed demonstrations that show recommended patterns for using SVX containers, iterators, and algorithms. They are collected in `examples/use_models` and can be run together from a single top-level module. Each sub-example is described separately below.

5.1 List

Source: `examples/use_models/list_example.svh`

Demonstrates the canonical forward-iteration use model for SVX containers. A vector of 20 random integers is built using `appendc()`. A `list_iterator` is then created and bound to the vector, and the standard idiom (`first()` / `at_end()` / `get()` / `next()`) is used to traverse and print all elements.

The vector is then sorted with `sort()`, and the same iterator is reset to `first()` and traversed again to display the sorted output. This example is the recommended starting point for understanding how to use SVX iterators.

SVX features exhibited:

- `vector#(T,P)`
- `list_iterator#(T,P)`
- `uint32_traits`

5.2 Map — Basic Use

Source: `examples/use_models/map_example.svh`

Demonstrates inserting, looking up, and iterating over a typed map. The map stores eight physical constants (Faraday, Planck, Avogadro, Boltzmann, electron mass, proton mass, magnetic flux constant, and elementary charge) keyed by their common names. A `map_iterator` traverses all entries, using `get_index()` to retrieve each key and printing the value and units.

Two direct lookups follow: one for a key that exists (`proton_mass`) and one for a key that does not (`alpha_particle_mass`), demonstrating the null-handle return on a failed `get()`.

SVX features exhibited:

- `map#(KEY,T,P)`
- `map_iterator#(KEY,T,P)`
- `class_traits#(T)`

5.3 Map — Polymorphic Container

Source: `examples/use_models/map_example.svh`

Demonstrates storing heterogeneous values of different types in a single map by wrapping them in `type_container#()`. Three entries are inserted under string keys: an integer, a real number, and a string, each wrapped in the corresponding `type_container` specialization.

The example then demonstrates two approaches to processing the polymorphic map. The first uses virtual dispatch: iterating the map and calling `convert2string()` on each `type_container_base` handle. The second uses type handles: each item's type is identified by comparing `get_type_handle()` against `type_handle#(T)::get_type()` in a case statement, allowing the handle to be cast to the concrete type without virtual functions.

SVX features exhibited:

- `map#(KEY,T,P)`
- `map_iterator#(KEY,T,P)`
- `type_container#(T)`
- `type_container_base`
- `type_handle#(T)`
- `class_traits#(T)`

5.4 Algo with Range

Source: `examples/use_models/algo_example.svh`

Demonstrates running algorithms over a sub-range of a vector using the `range#()` iterator. A vector of random 16-bit integers is created with a randomized size. A range iterator is then bound to a `list_iterator` over the vector, with randomly chosen lower and upper bounds.

`for_each` prints the full vector, then `for_each` prints just the elements within the range. `count` and `all_of` are run over the range using a predicate that tests whether a value is ≥ 50 , showing how `range#()` composes with the algorithm library to restrict operations to a sub-sequence.

SVX features exhibited:

- `algo#(T,P)`
- `predicate#(T)`
- `fcn#(T)`
- `vector#(T,P)`
- `list_iterator#(T,P)`
- `range#(T,P)`
- `uint16_traits`

Chapter 6: Applications

The application examples demonstrate SVX's three application-layer packages: `mem`, `mem_map`, and `pri_queue`. Each example is a complete, runnable simulation.

6.1 Sparse Memory

Source: `examples/mem`

Demonstrates the `mem#()` sparse memory model in a read/write verification scenario. A 32-bit memory with 16-bit pages, 8-bit blocks, and a 4-byte word size is instantiated. One word at a fixed offset within the test region is marked write-protected with `set_word_restriction()` before any writes begin.

100 random words are written to consecutive word-aligned addresses starting at a base address of `0x7460_0000`. After writing, all 100 locations are read back and compared against a local reference array. Data mismatches are reported as errors. The write to the protected address is also expected to fail; `last_operation_failed()` and `get_last_op_string()` detect and report the restriction violation. The test concludes with calls to `show()` and `dump()` to display the security map and allocated page contents.

SVX features exhibited:

- `mem#(ADDR_BITS, PAGE_BITS, BLOCK_BITS, WORD_SIZE)`

6.2 Device Memory Map

Source: `examples/mem_map`

Constructs a realistic SoC-style hierarchical address map and exercises its path-based and address-based lookup facilities. The hierarchy models a complete system with a 64 KB address space (`0x0000–0xFFFF`). An IO bus region at `0xFF00` contains two UART sub-regions, each with `read_buf`, `write_buf`, and `ctrl` registers. A system bus at `0x0000` is modeled by two overlapping views: `sys_bus`, which contains a timer region (with a CSR register holding `start_stop` and `status` fields) and two memory blocks; and `sys_bus_2`, an alternate view of the same address range that exposes the memory as a single flat block. The two views are both of type `mem_view#()`, which permits overlapping sibling instances.

After the hierarchy is constructed, `calculate_and_check()` is called on the root to propagate absolute addresses and verify that no non-view siblings overlap. `dump()` prints the entire hierarchy with offsets, sizes, and absolute addresses. Three path-based lookups using `find()` demonstrate successful and failed navigation by dot-separated name path. Six address-based lookups using `find_addr_all()` return all spaces at every hierarchy level whose range contains the queried address; an iterator over the returned list prints each matching space.

SVX features exhibited:

- `mem_region#(ADDR_SIZE)`
- `mem_view#(ADDR_SIZE)`
- `mem_register#(ADDR_SIZE)`
- `mem_field#(ADDR_SIZE)`
- `mem_memory#(ADDR_SIZE)`
- `mem_space#(ADDR_SIZE)`

6.3 Priority Queue Job Scheduler

Source: `examples/pri_queue`

Simulates a priority-based job scheduler using two concurrent simulation threads. Sixteen jobs labeled A through P are pre-loaded into an SVX queue, which serves as a staging buffer for convenience. The first thread dequeues jobs one at a time, assigns each a randomized priority (0–9), pushes it into a priority queue, and waits two time units before pushing the next.

The second thread runs concurrently, waiting for the priority queue to become non-empty, waiting five additional time units, and then popping and printing the highest-priority job available. Because the inter-push delay (2 units) is shorter than the inter-pop delay (5 units), the queue accumulates jobs before each pop, and the pop always retrieves the highest-priority job from among those currently waiting. This demonstrates that `pri_queue#()` maintains priority order dynamically as items arrive.

SVX features exhibited:

- `pri_queue#(T,P)`
- `queue#(T,P)`
- `string_traits`